

DietiEstates25

Documentazione

Progetto ING-SW – DietiEstates25
Università degli Studi di Napoli Federico II

15 febbraio 2026

Indice

1	Documento dei Requisiti Software	4
1.1	Contesto e obiettivi	4
1.2	Glossario	5
1.3	Personas	6
1.3.1	Persona 1 – Giulia	6
1.3.2	Persona 2 – Marco	7
1.3.3	Persona 3 – Assunta, agente immobiliare	8
1.3.4	Persona 4 – Franco, manager di agenzia	9
1.4	Attori e casi d’uso	10
1.4.1	Attori	10
1.4.2	Elenco principali casi d’uso	10
1.5	Requisiti funzionali	12
1.6	Requisiti non funzionali	12
1.7	Requisiti di dominio	13
1.8	MockUp dei casi d’uso	13
1.8.1	M0: Login Page	13
1.8.2	M1: Landing Page	14
1.8.3	M2: Pagina di Ricerca	14
1.8.4	M3: Ricerca Senza Risultati	15
1.8.5	M4: Dashboard	15
1.8.6	M5: Creazione Immobile	16
1.8.7	M6: Creazione Immobile - Campi Incompleti	17
1.8.8	M7: Pagina di Profilo	18
1.8.9	M8: Modifica in Corso	18
1.8.10	M9: Modifica con Successo	19
1.8.11	M10: Pagina di Dettaglio Agente	20
1.8.12	M11: Aggiunta Recensione	21
1.8.13	M12: Recensione Aggiunta	22
1.9	Formalizzazione di casi d’uso (Cockburn)	23

2	Documento di Design del sistema	27
2.1	Architettura	27
2.1.1	Strati logici back-end	27
2.1.2	Pattern architetturali e di design	28
2.1.3	Struttura front-end	28
2.2	Criteri di design	29
2.3	Tecnologie adottate	29
2.3.1	Back-end	29
2.3.2	Front-end	29
2.3.3	DevOps e strumenti	29
2.4	Deployment e infrastruttura	30
2.4.1	Back-end e database	30
2.4.2	Storage delle immagini	30
2.5	Schema di persistenza dati	31
2.6	Design dell'interfaccia utente	32
2.7	Diagramma delle classi di design	33
2.7.1	Diagrammi Architetturali	34
2.8	Diagrammi di sequenza	36
2.8.1	Sequence Diagram: UC06	36
2.8.2	Sequence Diagram: UC08	37
2.8.3	Sequence Diagram: UC10	38
2.8.4	Sequence Diagram: UC12	39
2.9	Gestione del versioning	40
2.9.1	Statistiche del repository	40
2.10	Analisi della qualità del codice	42
2.10.1	Report SonarQube	42
3	Testing e valutazione dell'usabilità	43
3.1	Strategia di unit testing	43
3.1.1	Classi di equivalenza e criteri di copertura	43
3.1.2	Esempi di metodi testati con classi di equivalenza	43
3.1.3	Verifica dei risultati e dello stato	46
3.1.4	Copertura del codice	47
3.2	Testing di integrazione e API	47
3.2.1	Test dei Controller	47
3.2.2	Criteri di copertura per i test di integrazione	48
3.2.3	Test di integrazione con database	48

3.3	Valutazione dell'usabilità	49
3.3.1	Expert reviews / inspections (Nielsen Heuristics)	49
3.3.2	Esperimento con utenti	49
3.3.3	Risultati	50
3.3.4	Conclusioni e raccomandazioni	51

Capitolo 1

Documento dei Requisiti Software

1.1 Contesto e obiettivi

DietiEstates25 è una piattaforma web per la gestione di servizi immobiliari multi-agenzia. L'obiettivo principale del sistema è:

- **Centralizzare** la gestione di agenzie, agenti, immobili e clienti.
- **Permettere agli utenti finali** di cercare immobili (vendita/affitto) con ricerca avanzata e mappa interattiva.
- **Supportare gli attori professionali** (agenzie, agenti) nella pubblicazione e gestione degli immobili.
- **Garantire sicurezza** delle credenziali e una *user experience* moderna, intuitiva e responsiva.

Lo stato attuale dell'implementazione copre:

- Autenticazione e registrazione (utente, agenzia, agenti).
- Gestione agenzie e agenti (creazione agenti da parte del manager dell'agenzia).
- Gestione immobili (creazione annuncio con dettagli completi, coordinate geografiche, foto, servizi).
- Ricerca avanzata con filtro su parametri multipli e supporto ricerca geografica per raggio.
- Profili agenti con biografia, foto, proprietà collegate e recensioni clienti.
- Gestione profilo utente (dati personali, password, foto profilo per agenti).
- Recensioni agenti (rating numerico + commento testuale).
- Login/registrazione con provider esterni (Google, Facebook, GitHub).

Funzionalità previste dal capitolato ma **non ancora implementate** (o pianificate come estensione futura):

- Prenotazione visite agli immobili con agenda condivisa.
- Gestione di offerte di acquisto/affitto (workflow proposta/controproposta).
- Tipologie dedicate per affitti brevi / case vacanze.

1.2 Glossario

Termine	Definizione
Agenzia	Organizzazione immobiliare che utilizza la piattaforma. Modellata come entità Agency nel back-end.
Manager di agenzia	Utente che rappresenta il gestore dell'agenzia; registra l'agenzia e crea gli account degli agenti.
Agente	Professionista immobiliare associato ad una Agenzia, modellato come sottoclasse Agent di User .
Utente (cliente)	Utente registrato interessato alla ricerca di immobili (acquisto/affitto), con ruolo USER .
Account esterno	Account OAuth2 (Google, Facebook, GitHub) collegato all'utente; indicato dal campo authProvider .
Immobile / Property	Entità Property : rappresenta un annuncio immobiliare (titolo, descrizione, prezzo, parametri fisici, ecc.).
Annuncio	Sinonimo operativo di "Immobile pubblicato" sulla piattaforma.
Amenity / Servizio	Servizio/comfort associato a un immobile (Amenity : es. Wi-Fi, portineria, climatizzazione, garage).
Classe energetica	Indicatore di efficienza energetica (EnergyClass), memorizzato come enum.
Recensione	Valutazione di un agente da parte di un utente, con punteggio numerico e commento (Review).
Profilo agente	Vista pubblica con dati dell'agente: biografia, foto, recapiti, lista immobili, recensioni.
Profilo utente	Vista privata dove l'utente modifica nome, cognome, e-mail, password e (per agenti) foto/biografia.
Ricerca avanzata	Funzionalità di filtraggio immobili per parametri multipli (prezzo, stanze, città, classe energetica, ecc.).
Ricerca geografica	Ricerca immobili entro un raggio da un punto selezionato su mappa (latitudine/longitudine + raggio).
JWT	JSON Web Token usato per autenticazione stateless tra front-end Angular e API Spring Boot.
H2	Database in-memory usato in sviluppo, configurato in modalità PostgreSQL.
S3 Storage	Sistema di storage (AWS S3) per file/foto, incapsulato in StorageService .

Tabella 1.1: Glossario dei principali termini di dominio

1.3 Personas

1.3.1 Persona 1 – Giulia



Giulia Panini

Profilo:

Giulia ha 32 anni, lavora come impiegata amministrativa in un'azienda di servizi a Napoli. Vive in affitto da 5 anni e ora vuole comprare casa. Ha un reddito stabile e un budget definito. È abituata a utilizzare servizi online e preferisce informarsi online prima di prendere decisioni importanti. Si aspetta dalla piattaforma informazioni chiare, foto di qualità e la possibilità di valutare l'affidabilità degli agenti.

Obiettivi:

- Trovare un appartamento in vendita a Napoli in zona semi-centrale.
- Filtrare per budget, numero stanze, classe energetica.
- Valutare rapidamente l'affidabilità dell'agente tramite recensioni.

Motivazioni:

Vuole ridurre il tempo speso in visite inutili, preferisce interfacciarsi con agenti trasparenti e affidabili, apprezza informazioni chiare e complete prima di contattare.

Competenze digitali:

Medio-alte. Naviga con sicurezza, utilizza filtri avanzati, si orienta su mappe interattive.

Scenari tipici:

- Affina la ricerca con slider prezzo/superficie e filtri aggiuntivi.
- Consulta la pagina dell'agente e le recensioni di altri clienti prima di contattarlo.

Pain points:

- Difficoltà a confrontare rapidamente più immobili.
- Mancanza di informazioni sulla classe energetica o sui costi condominiali.
- Agenti poco reattivi o con recensioni negative non facilmente identificabili.

1.3.2 Persona 2 – Marco



Marco Grimaldi

Profilo:

Marco ha 24 anni, ha recentemente completato la sua formazione da fotografo e sta cominciando a lavorare. “È fuori sede e cerca casa in affitto per iniziare la carriera. Ha un budget limitato (circa 500–600€/mese) e poco tempo disponibile per visite.

Obiettivi:

- Trovare un bilocale in affitto vicino all’università o al posto di lavoro.
- Filtrare per prezzo massimo e presenza di servizi essenziali.
- Valutare rapidamente se l’immobile soddisfa le esigenze di base.
- Evitare perdite di tempo con annunci fuori budget o in zone scomode.

Motivazioni:

Budget limitato, necessità di ottimizzare le risorse, poco tempo disponibile per visite multiple, priorità a funzionalità essenziali rispetto a lussi.

Competenze digitali:

Buone, ma poca tolleranza per interfacce complicate o lente. Preferisce soluzioni immediate e intuitive.

Scenari tipici:

- Usa la barra di ricerca in home (“Napoli”) + tipo “RENT”.
- Utilizza il raggio sulla mappa per delimitare una zona di interesse specifica.
- Seleziona immobili con foto chiare e descrizione dettagliata.
- Controlla rapidamente la presenza di servizi essenziali prima di contattare l’agente.

Pain points:

- Annunci senza indicazione chiara del prezzo mensile.
- Mappe poco chiare o difficili da usare per delimitare zone specifiche,
- Processo di ricerca troppo lungo o con troppi passaggi.

1.3.3 Persona 3 – Assunta, agente immobiliare



Assunta Russo

Profilo:

Assunta ha 40 anni, lavora come agente immobiliare da oltre 15 anni in un'agenzia locale di medie dimensioni a Napoli. Gestisce un portafoglio di circa 20–30 immobili tra vendita e affitto. Utilizza PC e smartphone per lavoro quotidiano ma non è esperta di tecnologie avanzate. Vuole pubblicare annunci rapidamente e mantenere un profilo professionale che ispiri fiducia. Si aspetta dalla piattaforma un'interfaccia semplice e upload foto veloce.

Obiettivi:

- Pubblicare rapidamente nuovi immobili con foto e dettagli completi.
- Dare un'immagine professionale tramite profilo con foto e biografia.
- Ricevere recensioni positive dai clienti per aumentare la credibilità.
- Monitorare le performance degli annunci pubblicati.

Motivazioni:

Aumentare la visibilità degli immobili dell'agenzia, aumentare la fiducia dei clienti nel proprio operato, ridurre il tempo necessario per pubblicare un nuovo annuncio.

Competenze digitali:

Medie. Utilizza PC e smartphone per lavoro, ma non è esperta di tecnologie avanzate. Preferisce interfacce semplici e intuitive.

Scenari tipici:

- Crea un nuovo annuncio immobile.
- Aggiorna la propria biografia e carica una foto profilo professionale.
- Monitora le recensioni ricevute.

Pain points:

- Processo di upload foto troppo lento o complicato.
- Difficoltà a posizionare correttamente l'immobile sulla mappa.

1.3.4 Persona 4 – Franco, manager di agenzia



Franco Ferrara

Profilo:

Franco ha 50 anni, è titolare di un'agenzia immobiliare di medie dimensioni a Napoli con 5–8 agenti. Gestisce sia la parte operativa che quella amministrativa. Ha iniziato l'attività oltre 20 anni fa e cerca soluzioni digitali per modernizzare i processi senza stravolgere il lavoro quotidiano. Vuole strumenti semplici, affidabili e che riducano il carico amministrativo. Si aspetta dalla piattaforma un processo di registrazione chiaro e una dashboard semplice per gestire gli agenti.

Obiettivi:

- Registrare l'agenzia sulla piattaforma e configurare l'account iniziale.
- Creare account per i propri agenti in modo semplice e veloce.
- Monitorare il portafoglio immobili pubblicati dall'agenzia.
- Verificare le performance degli agenti (numero di annunci, recensioni ricevute).

Motivazioni:

Centralizzare i processi digitali dell'agenzia, migliorare la qualità del servizio verso i clienti, aumentare la visibilità complessiva dell'agenzia sul mercato.

Competenze digitali:

Medie. Utilizza computer per lavoro quotidiano, ma non è esperto di tecnologie avanzate. Preferisce soluzioni semplici e affidabili.

Scenari tipici:

- Registra l'agenzia tramite form dedicato con dati completi.
- Accede alla dashboard agenzia e visualizza panoramica degli agenti.
- Crea nuovi account agenti inserendo dati anagrafici e di contatto.

Pain points:

- Processo di registrazione iniziale troppo complesso o poco chiaro.
- Mancanza di dashboard chiara per monitorare le attività dell'agenzia.

1.4 Attori e casi d'uso

1.4.1 Attori

- **Visitatore:** utente non autenticato che può cercare e visualizzare immobili e profili agenti.
- **Cliente:** utente registrato interessato a servizi immobiliari.
- **Agente immobiliare:** utente con ruolo professionale associato a un'agenzia.
- **Manager di agenzia:** utente che registra e gestisce un'agenzia e i relativi agenti.
- **Provider OAuth2 esterno:** Google, Facebook, GitHub.

1.4.2 Elenco principali casi d'uso

- UC01 – Registrazione utente.
- UC02 – Login utente.
- UC03 – Login con account esterno (Google/Facebook/GitHub).
- UC04 – Registrazione di una nuova agenzia.
- UC05 – Creazione account agente da parte del manager.
- UC06 – Ricerca avanzata di immobili (filtri + mappa).
- UC07 – Visualizzazione dettaglio immobile.
- UC08 – Creazione di un nuovo immobile (agente).
- UC09 – Gestione profilo utente.
- UC10 – Gestione profilo agente (biografia, foto profilo).
- UC11 – Visualizzazione profilo agente con immobili e recensioni.
- UC12 – Inserimento recensione su un agente.
- UC13 – Visualizzazione elenco agenti.

Use Case Diagrams

I casi d'uso sopra elencati sono modellati tramite Use Case Diagram UML:

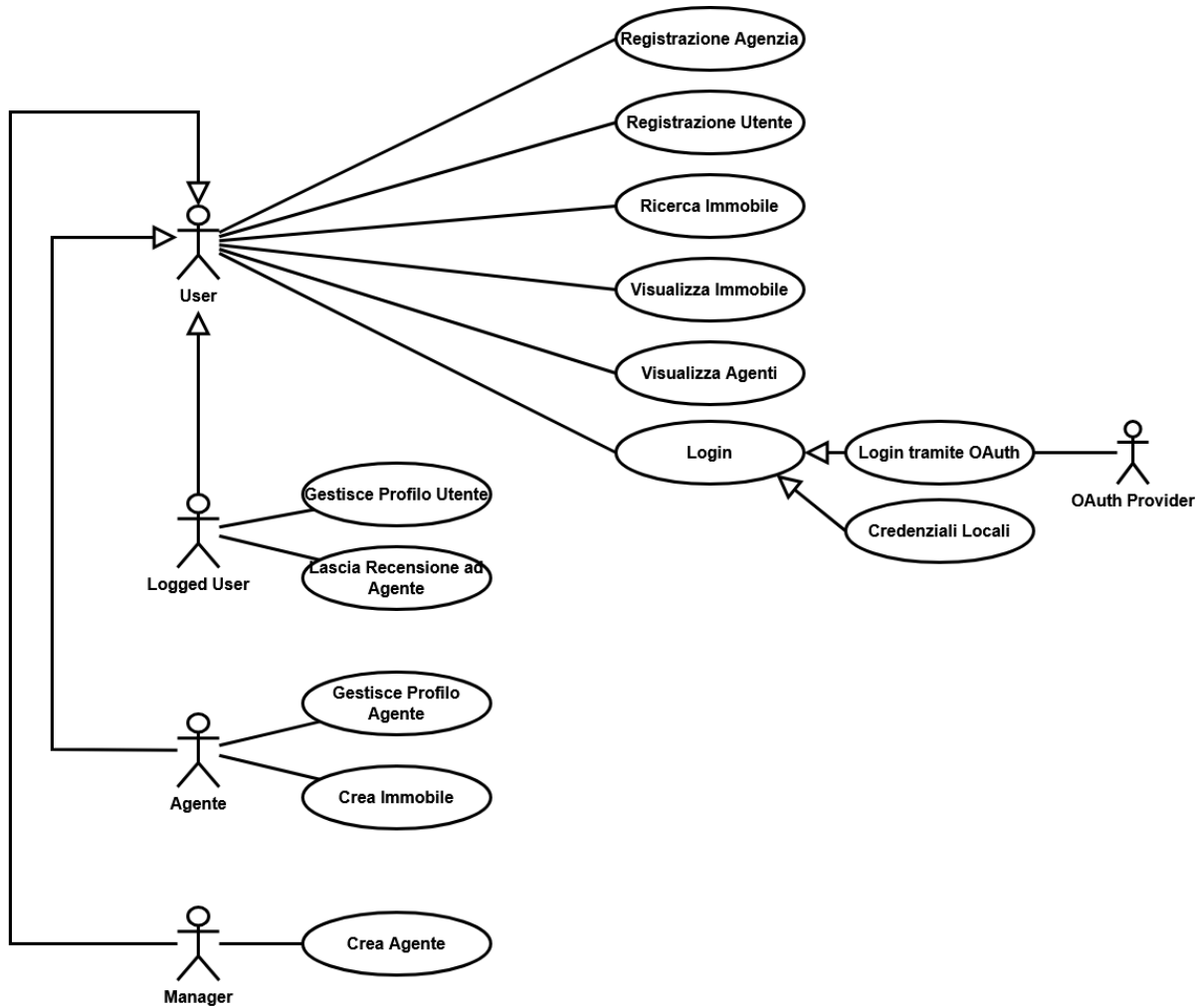


Figura 1.1: Use Case Diagram

1.5 Requisiti funzionali

- **RF-01 – Registrazione utente cliente**
Un utente può registrarsi con e-mail e password (endpoint `/auth/register`).
- **RF-02 – Registrazione agenzia + manager**
Un manager registra l'agenzia e il proprio account manager (endpoint `/auth/register/agency`).
- **RF-03 – Login con credenziali locali**
Login con e-mail/password, ricezione JWT, accesso ad API protette (endpoint `/auth/login`).
- **RF-04 – Login tramite provider esterno**
Accesso tramite Google, Facebook, GitHub (flusso OAuth2 configurato in `application.properties`).
- **RF-05 – Creazione account agente**
Il manager di agenzia crea account agenti associati alla propria agenzia.
- **RF-06 – Creazione annuncio immobile**
Un agente autenticato crea un immobile completo di dettagli, servizi, foto e posizione.
- **RF-07 – Ricerca avanzata immobili**
L'utente può cercare immobili usando filtri multipli: città, tipo, prezzo, stanze, metri quadri, piano, bagni, classe energetica, agente.
- **RF-08 – Ricerca geografica a raggio**
L'utente può selezionare un punto sulla mappa e un raggio per limitare la ricerca.
- **RF-09 – Visualizzazione immobili su mappa**
I risultati vengono mostrati anche su una mappa interattiva (Google Maps).
- **RF-10 – Gestione profilo utente**
L'utente autenticato modifica dati personali e password.
- **RF-11 – Gestione profilo agente**
L'agente aggiorna biografia e foto profilo, oltre ai propri dati personali.
- **RF-12 – Visualizzazione profilo agente**
L'utente consulta dati agenti, immobili associati e recensioni.
- **RF-13 – Inserimento recensione agente**
Un cliente autenticato inserisce una recensione (punteggio + commento) per un agente.

1.6 Requisiti non funzionali

- **RNF-01 – Prestazioni**
Le ricerche immobili devono completarsi in meno di un secondo dataset realistici.
- **RNF-02 – Scalabilità**
Architettura back-end stateless, sessioni gestite tramite JWT, possibilità di scalare orizzontalmente.
- **RNF-03 – Sicurezza**
Password memorizzate come hash sicuri; JWT firmati con chiave segreta; integrazione OAuth2; ruoli e autorizzazioni.
- **RNF-04 – Affidabilità**
Gestione centralizzata delle eccezioni; presenza di test unitari e di integrazione.
- **RNF-05 – Usabilità**
UI responsiva, flussi chiari per ricerca, login, gestione profilo; messaggi di errore espliciti.
- **RNF-06 – Manutenibilità**
Separazione in livelli (Controller, Service, Repository, DTO, Model); uso di convenzioni standard.
- **RNF-07 – Portabilità**
Utilizzo di Docker; dipendenze configurabili tramite variabili d'ambiente.

1.7 Requisiti di dominio

- Supporto a tipologie di inserzione **SALE** e **RENT**, con previsione di estensione a affitti brevi/case vacanze.
- Ogni immobile è associato a un agente, un'agenzia, servizi (amenities), foto e ad una posizione geografica (punto).
- Gli utenti possono avere più ruoli; agenti e manager sono associati a un'agenzia.
- Le recensioni sono legate a un singolo agente e, opzionalmente, a un utente cliente.

1.8 MockUp dei casi d'uso

Di seguito sono riportati i mock up per i casi di uso e le loro estensioni.

1.8.1 M0: Login Page

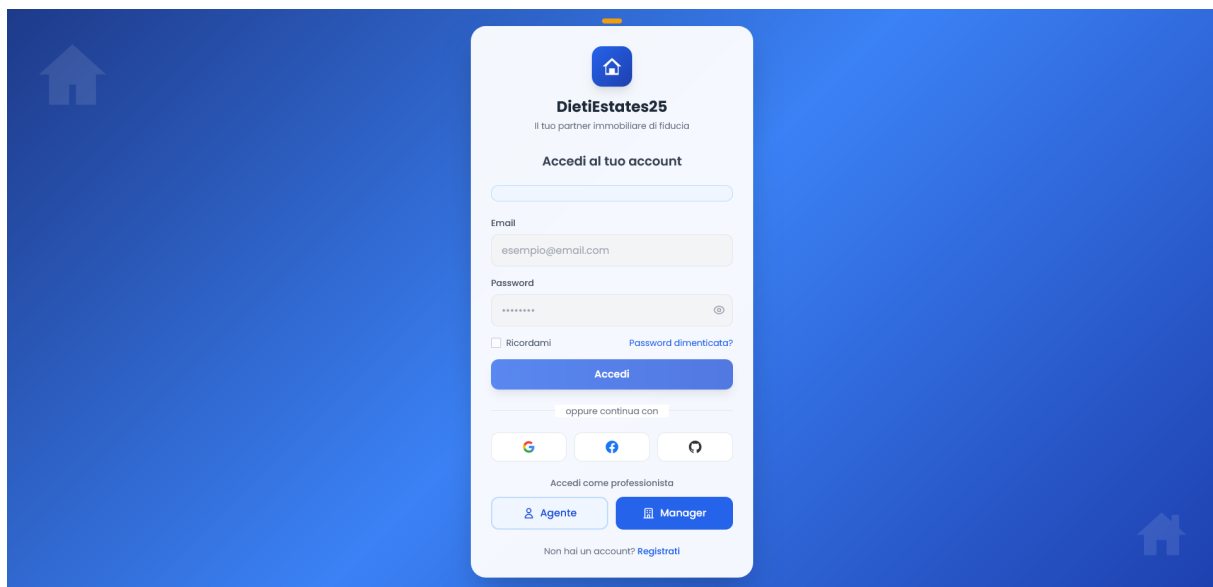


Figura 1.2: Login Page

1.8.2 M1: Landing Page



Figura 1.3: Landing Page

1.8.3 M2: Pagina di Ricerca

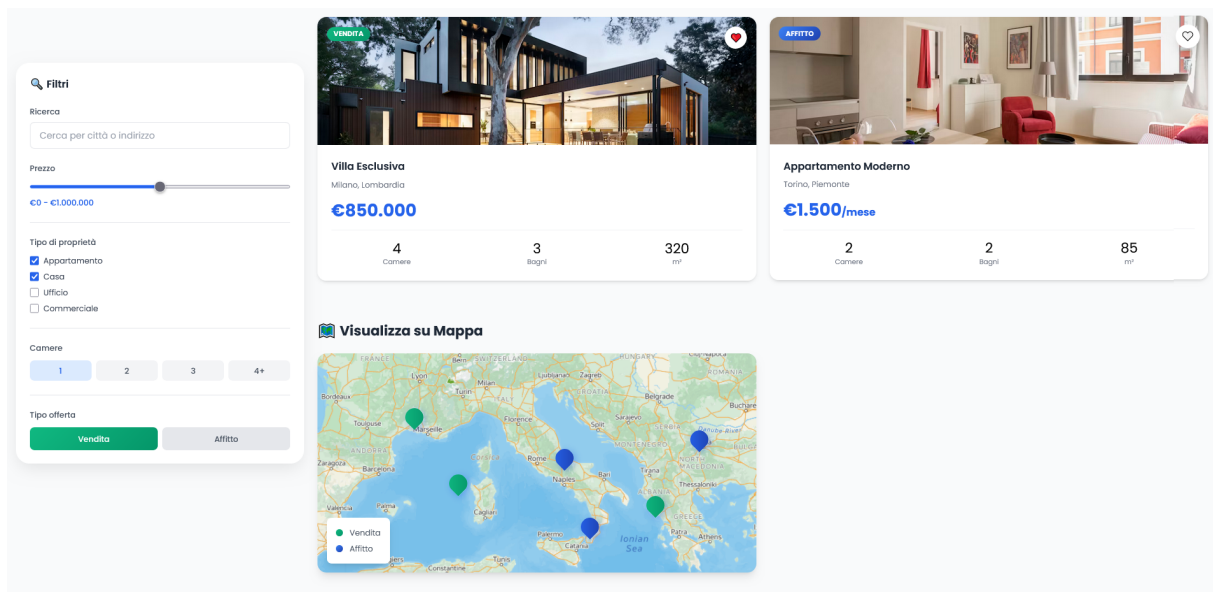


Figura 1.4: Pagina di Ricerca

1.8.4 M3: Ricerca Senza Risultati

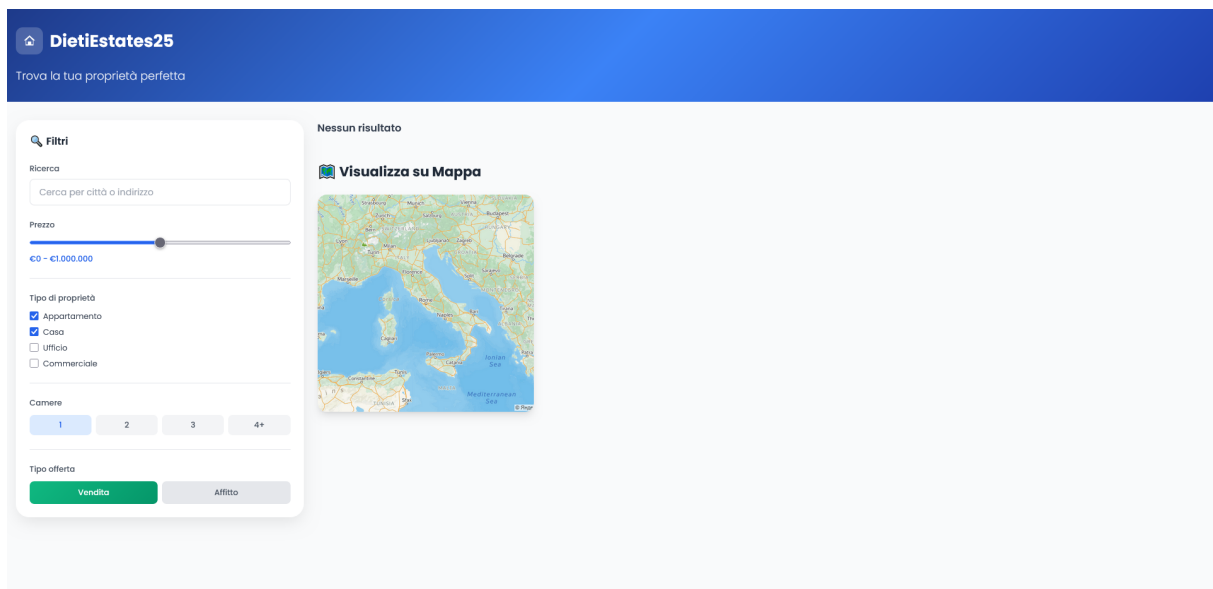


Figura 1.5: Ricerca Senza Risultati

1.8.5 M4: Dashboard

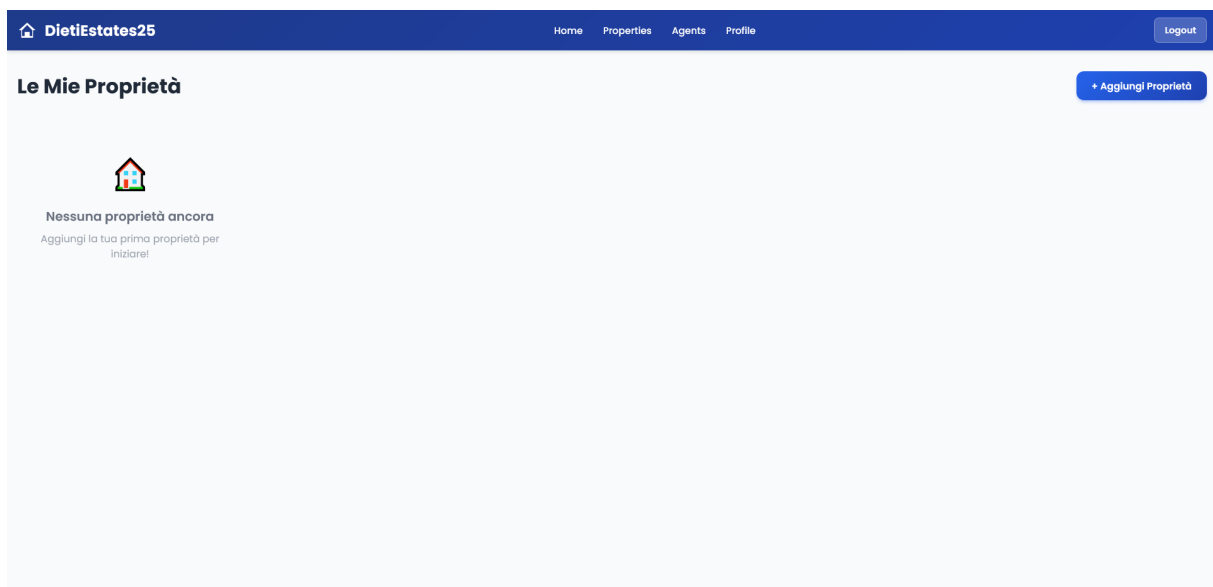


Figura 1.6: Dashboard

1.8.6 M5: Creazione Immobile

**DietiEstates25**

Crea Nuovo Immobile

 **Informazioni Generali**

Titolo Immobile *

es. Villa Esclusiva

Tipo di Proprietà *

Seleziona tipo

Tipo di Offerta *

Seleziona offerta

 **Localizzazione**

Indirizzo *

es. Via Roma, 123

Città *

es. Milano

Provincia *

es. Lombardia

 **Dettagli Proprietà**

Prezzo *

850000

€

Camere *

4

Bagni *

3

Superficie (m²) *

320

 **Descrizione**

Descrizione Immobile

Descrivi le caratteristiche e lo stato...

 **Caratteristiche**

☐ Parcheggio

☐ Giardino

☐ Piscina

☐ Balcone

☐ Ascensore

☐ A/C

Salva Immobile

* Campi obbligatori

Figura 1.7: Creazione Immobile

1.8.7 M6: Creazione Immobile - Campi Incompleti

 **DietiEstates25**

Crea Nuovo Immobile

Completare tutti i campi obbligatori per salvare correttamente l'immobile

 **Informazioni Generali**

Titolo Immobile *

es. Villa Esclusiva

Tipo di Proprietà *

Seleziona tipo

Tipo di Offerta *

Seleziona offerta

 **Localizzazione**

Indirizzo *

es. Via Roma, 123

Città *

es. Milano

Provincia *

es. Lombardia

 **Dettagli Proprietà**

Prezzo *

850000

€

Camere *

4

Bagni *

3

Superficie (m²) *

320

 **Descrizione**

Descrizione Immobile

Descrivi le caratteristiche e lo stato...

 **Caratteristiche**

☐ Parcheggio

☐ Giardino

☐ Piscina

☐ Balcone

☐ Ascensore

☐ A/C

Salva Immobile

* Campi obbligatori

Figura 1.8: M5: Creazione Immobile - Campi Incompleti

1.8.8 M7: Pagina di Profilo

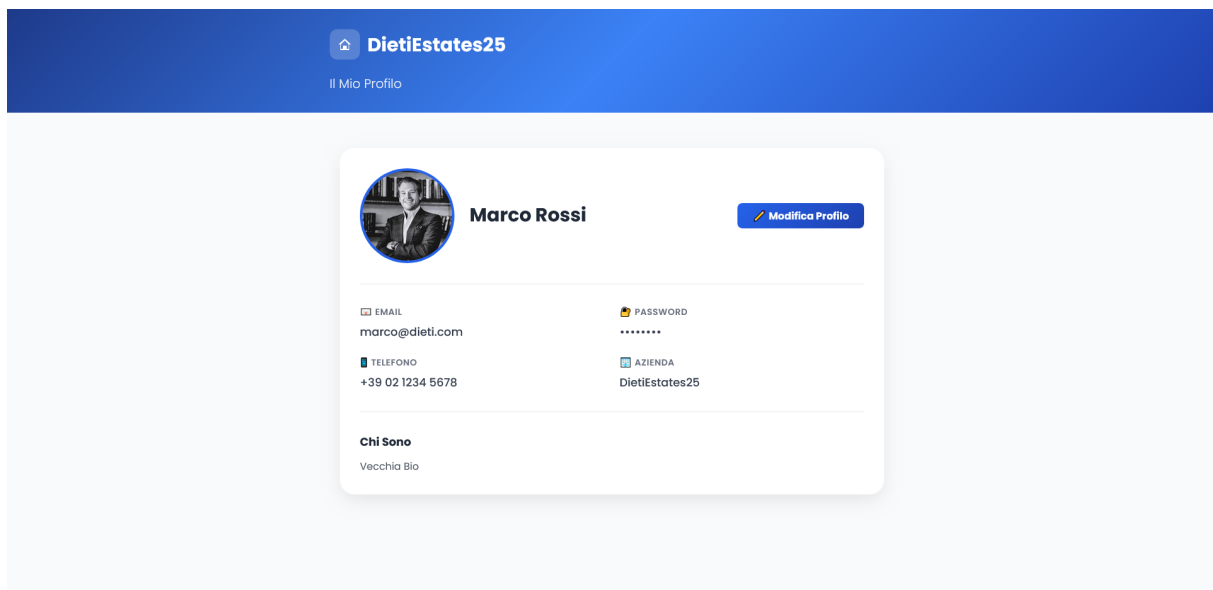


Figura 1.9: Pagina di Profilo

1.8.9 M8: Modifica in Corso

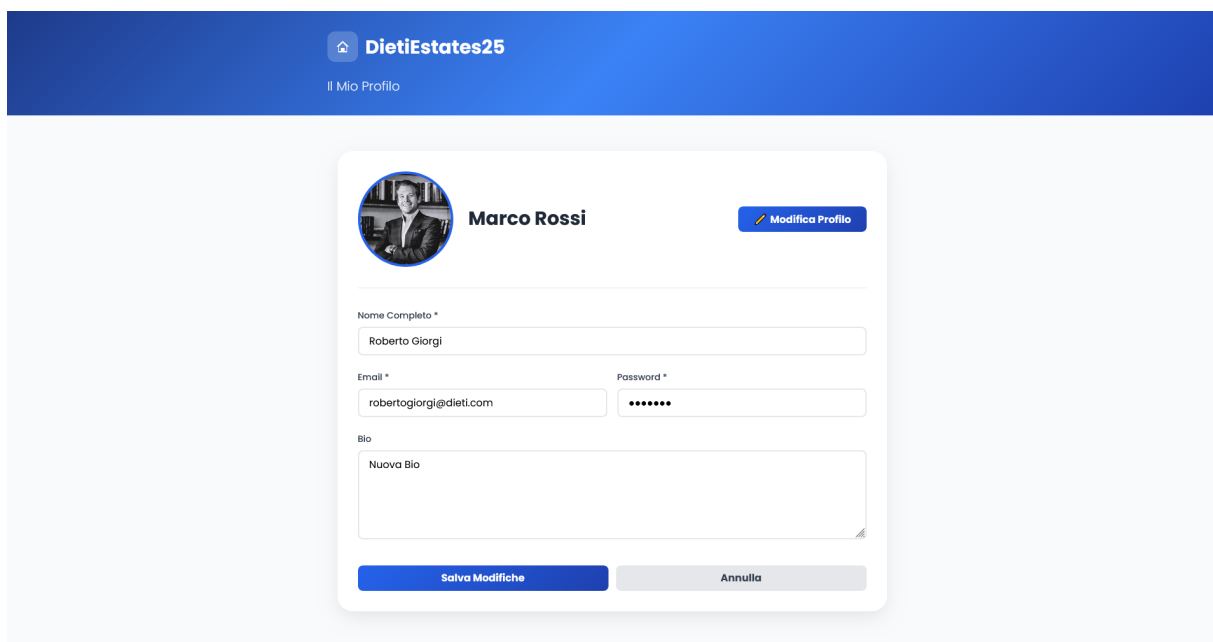


Figura 1.10: Modifica in Corso

1.8.10 M9: Modifica con Successo

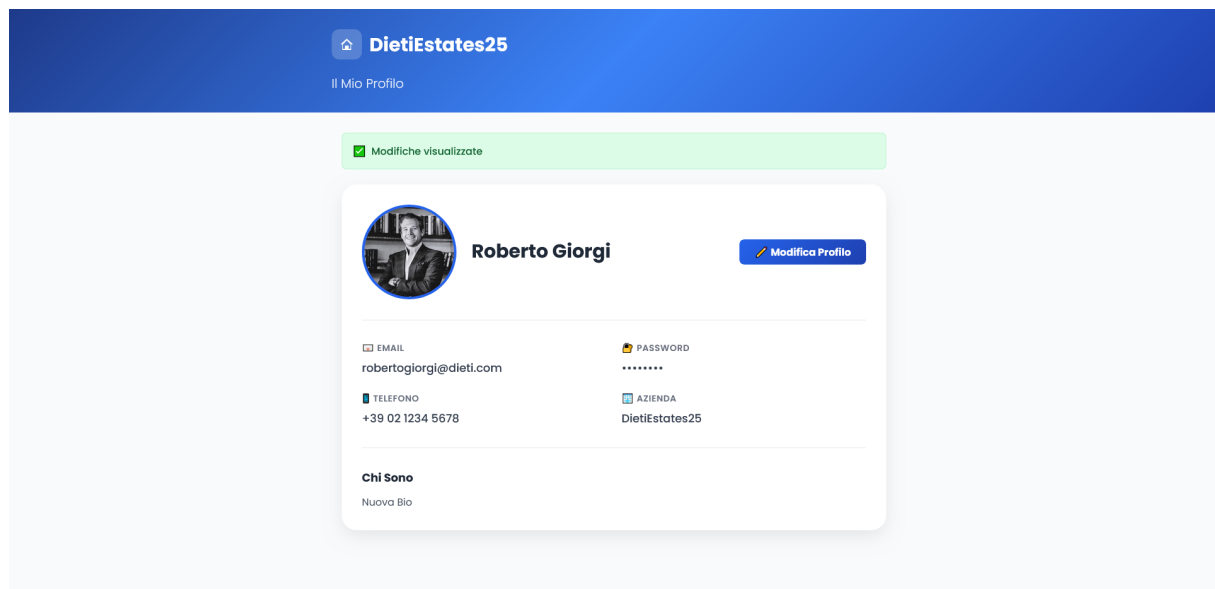


Figura 1.11: Modifica con Successo

1.8.11 M10: Pagina di Dettaglio Agente

 DietiEstates25



Andrea Speranza
Consulente Immobiliare
★★★★★ (156 recensioni)
Valutatore certificato con una lunga carriera nel settore. Massima trasparenza e professionalità.
[+39 02 3456 7890](tel:+390234567890) luca@dieti.it

Proprietà Gestite (8)



Villa Esclusiva Milano
Via Roma, Milano
€ 850.000
4 camere • 300 m²



Appartamento Centro
Corso Vittorio, Milano
€ 450.000
3 camere • 150 m²



Penthouse Duomo
Piazza Duomo, Milano
€ 1.200.000
5 camere • 400 m²



Ufficio Navigli
Navigli, Milano
€ 320.000
250 m² • Moderno

Recensioni (156) [Aggiungi Recensione](#)

Marco Bianchi ★★★★★
Andrea è stato fantastico! Mi ha aiutato a vendere la mia proprietà in record time. Professionista serio e disponibile.
2 settimane fa

Giulia Rossi ★★★★★
Esperienza eccezionale! Andrea conosce il mercato perfettamente e i suoi consigli sono sempre molto utili e preziosi.
1 mese fa

Antonio Ferrari ★★★★★
Buon professionista. Un po' di attesa nel processo finale, ma comunque consigliato.
1 mese fa

Figura 1.12: Pagina di Dettaglio Agente

1.8.12 M11: Aggiunta Recensione

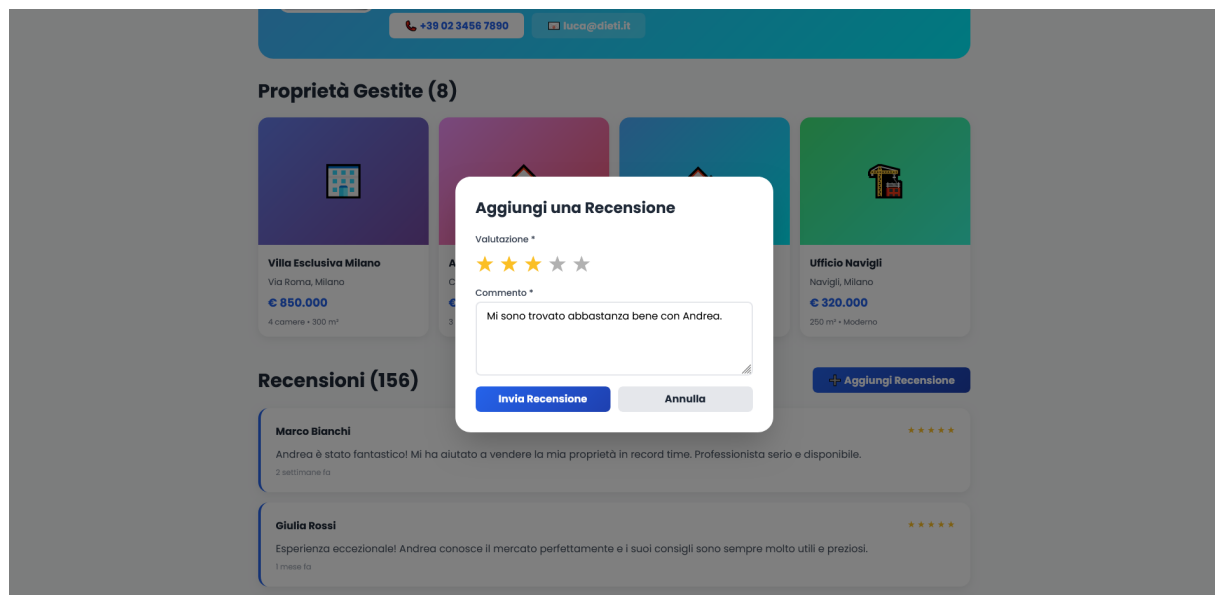


Figura 1.13: Aggiunta Recensione

1.8.13 M12: Recensione Aggiunta

 DietiEstates25



Andrea Speranza
Consulente Immobiliare
★★★★★ (156 recensioni)
Valutatore certificato con una lunga carriera nel settore. Massima trasparenza e professionalità.
[+39 02 3456 7890](tel:+390234567890) luca@dieti.it

Recensione lasciata con successo! (demo)

Proprietà Gestite (8)



Villa Esclusiva Milano
Via Roma, Milano
€ 850.000
4 camere • 300 m²



Appartamento Centro
Corso Vittorio, Milano
€ 450.000
3 camere • 150 m²



Penthouse Duomo
Piazza Duomo, Milano
€ 1.200.000
5 camere • 400 m²



Ufficio Navigli
Navigli, Milano
€ 320.000
250 m² • Moderno

Recensioni (156) [+ Aggiungi Recensione](#)

Marco Bianchi ★★★★★
Andrea è stato fantastico! Mi ha aiutato a vendere la mia proprietà in record time. Professionista serio e disponibile.
2 settimane fa

Giulia Rossi ★★★★★
Esperienza eccezionale! Andrea conosce il mercato perfettamente e i suoi consigli sono sempre molto utili e preziosi.
1 mese fa

Antonio Ferrari ★★★★★
Buon professionista. Un po' di attesa nel processo finale, ma comunque consigliato.
1 mese fa

Figura 1.14: Recensione Aggiunta

1.9 Formalizzazione di casi d'uso (Cockburn)

Di seguito quattro casi d'uso significativi (esclusi quelli di registrazione/autenticazione), formalizzati secondo Cockburn.

UC06 – Ricerca avanzata di immobili

USE CASE #06	Ricerca avanzata immobili		
Goal in Context	Trovare immobili con filtri e mappa.		
Preconditions	DB popolato, servizi attivi.		
Success End Cond.	Lista immobili e marker su mappa visualizzati.		
Failed End Cond.	Nessun immobile trovato.		
Primary Actor	Utente (Visitatore/Cliente)		
Trigger	Click su "Properties" nella Navbar.		
Main Scenario	Step	Actor Action	System Action
	1	Va su M1 e preme "Properties".	
	2		Mostra M2 (Ricerca) con filtri default.
	3	Imposta filtri e preme "Cerca".	
	4		Mostra risultati su lista e mappa in M2 .
Extensions	Step	Condition	System Action
	4a	Nessun risultato trovato.	Mostra M3 (Ricerca Senza Risultati).

Tabella 1.2: UC06 - Ricerca Immobili

UC08 – Creazione di un nuovo immobile (agente)

USE CASE #08	Creazione annuncio immobile		
Goal in Context	Agente pubblica nuovo immobile.		
Preconditions	Login effettuato come Agente.		
Success End Cond.	Immobile salvato e visibile.		
Failed End Cond.	Salvataggio fallito (dati incompleti).		
Primary Actor	Agente Immobiliare		
Trigger	Click su "Dashboard" nella Navbar.		
Main Scenario	Step	Actor Action	System Action
	1	Dalla M1 , va alla Dashboard.	Mostra M4 (Dashboard).
	2	Preme "Aggiungi Proprietà".	Mostra M5 (Creazione).
	3	Compila campi e preme "Salva".	
	4		Valida e salva. Mostra M4 aggiornata.
Extensions	Step	Condition	System Action
	3a	Campi obbligatori vuoti.	Mostra M6 (Campi Incompleti) ed errori.

Tabella 1.3: UC08 - Creazione Immobile

UC10 – Gestione profilo agente (biografia e foto)

USE CASE #10	Gestione profilo agente		
Goal in Context	Aggiornamento dati e biografia.		
Preconditions	Login effettuato come Agente.		
Success End Cond.	Dati aggiornati a sistema.		
Failed End Cond.	Modifiche non salvate.		
Primary Actor	Agente Immobiliare		
Trigger	Click su "Profilo" nella Navbar.		
Main Scenario	Step	Actor Action	System Action
	1	Preme "Profilo".	Mostra M7 (Profilo).
	2	Preme "Modifica".	Mostra M8 (Modifica in Corso).
	3	Modifica dati e preme "Salva".	
	4		Salva e mostra M9 (Successo).
Extensions	(Nessuna estensione critica prevista)		

Tabella 1.4: UC10 - Gestione Profilo

UC12 – Inserimento recensione agente

USE CASE #12	Inserimento recensione agente		
Goal in Context	Valutare agente con voto e testo.		
Preconditions	Utente loggato.		
Success End Cond.	Recensione salvata.		
Failed End Cond.	Recensione non inviata.		
Primary Actor	Utente Registrato		
Trigger	Selezione agente dalla lista.		
Main Scenario	Step	Actor Action	System Action
	1	Seleziona un agente.	Mostra M10 (Dettaglio Agente).
	2	Preme "Aggiungi Recensione".	Mostra form M11 .
	3	Inserisce dati e preme "Invia".	
	4		Registra e mostra M12 (Review Aggiunta).
Extensions	(Nessuna estensione critica prevista)		

Tabella 1.5: UC12 - Inserimento Recensione

Capitolo 2

Documento di Design del sistema

2.1 Architettura

L'architettura globale è di tipo client-server:

- **Front-end:** Single Page Application Angular 21.
- **Back-end:** REST API Spring Boot 3.2.
- **Database:** H2 in sviluppo (modalità PostgreSQL), PostgreSQL/PostGIS in produzione.
- **Storage file:** AWS S3 (o storage locale) incapsulato in un servizio di storage.
- **Autenticazione:** JWT per le API; OAuth2 per login sociale.

2.1.1 Strati logici back-end

Il back-end adotta una architettura **a tre livelli** (Controller–Service–Repository), tipica dei sistemi enterprise basati su Spring. Questo pattern separa nettamente:

- la gestione delle richieste HTTP,
- la logica di dominio e di business,
- l'accesso ai dati persistenti.

In dettaglio:

- **Presentation layer (Controller):**
 - espone endpoint REST e mappa le richieste HTTP verso operazioni di business;
 - valida i dati di input a livello di request (es. annotazioni `@Valid` su DTO di ingresso);
 - traduce le entità/DTO interni in risposte HTTP (codici di stato, payload JSON).
- **Business layer (Service):**
 - contiene la logica di dominio (es. regole di creazione immobili, associazione ad agenti e agenzie, calcolo e aggregazione delle recensioni);
 - coordina l'accesso ai repository e ad altri servizi (es. `StorageService`, `JwtService`, servizi di notifica);
 - rappresenta il *punto di ingresso* per i casi d'uso applicativi (UC06, UC08, UC10, UC12), rendendo più semplice il testing unitario delle regole di business.

2.1.2 Pattern architetturali e di design

Nell'implementazione di DietiEstates25 sono stati applicati diversi pattern architetturali e di design:

- **Layered Architecture:** l'organizzazione in livelli Controller–Service–Repository segue un'architettura stratificata, che separa presentazione, logica di business e accesso ai dati.
- **Repository Pattern:** le interfacce `*Repository` incapsulano le operazioni di persistenza sulle entità di dominio, fornendo un'API ad alto livello indipendente dal database sottostante (H2 in sviluppo, PostgreSQL/PostGIS in produzione).
- **DTO (Data Transfer Object):** l'uso di `*DTO` per richieste e risposte REST separa il modello interno dal contratto esterno delle API, riducendo il coupling e facilitando l'evoluzione del dominio e della versione delle API.
- **Dependency Injection / Inversion of Control:** i componenti Spring (controller, servizi, repository) e i servizi Angular sono istanziati e collegati dal container IoC, migliorando testabilità e sostituibilità delle dipendenze (ad esempio, `StorageService` può essere sostituito con uno stub per i test).

Questi pattern supportano gli obiettivi di **manutenibilità**, **riusabilità** e **scalabilità** definiti nei requisiti non funzionali, e rendono l'architettura coerente con le best practice dei framework utilizzati (Spring Boot e Angular).

- **Data access layer (Repository):**
 - astrae le operazioni CRUD sul database tramite Spring Data JPA;
 - definisce metodi di query ad alto livello (es. `findByCityAndPriceBetween(...)` o query personalizzate per la ricerca geografica);
 - isola il resto dell'applicazione dai dettagli del database (PostgreSQL/H2, PostGIS).

Questa suddivisione:

- migliora la **manutenibilità** (i cambiamenti in un livello impattano minimamente sugli altri);
- favorisce la **testabilità** (i service possono essere testati isolando i repository tramite mock);
- rende l'architettura **estendibile**, permettendo l'aggiunta di nuovi use case o integrazioni senza modificare i controller esistenti.

2.1.3 Struttura front-end

- Cartella `features/`:
 - **auth:** login, registrazione.
 - **properties:** lista, dettaglio, creazione.
 - **agents:** lista, dettaglio, dashboard agente.
 - **agency:** dashboard manager e creazione agenti.
 - **profile:** gestione profilo utente/agente.
 - **home:** pagina principale con ricerca.
- Cartella `core/`: servizi, modelli, interceptor JWT.
- Cartella `shared/`: componenti riutilizzabili (navbar, property-card, map-search, dual-range-slider, toast, footer).

2.2 Criteri di design

- **Separation of Concerns:** controller sottili, logica concentrata nei servizi.
- **DTO e Request objects:** per separare il modello interno dal contratto esterno delle API.
- **Security by design:** filtri JWT, configurazione fine-grained di ruoli e permessi.
- **Configurazione esterna:** parametri sensibili (JWT secret, chiavi OAuth, AWS) in variabili d'ambiente.
- **Estendibilità:** uso di enum e astrazioni di servizio per supportare facilmente nuove tipologie e provider.

2.3 Tecnologie adottate

2.3.1 Back-end

- **Spring Boot 3.2.1** (web, data-jpa, validation, security, oauth2-client).
- **Hibernate + Hibernate Spatial** per persistenza e tipi spaziali.
- **Spring Security + JWT** per autenticazione stateless.
- **H2** in-memory per sviluppo; **PostgreSQL/PostGIS** per produzione.
- **Lombok** per ridurre boilerplate.
- **SpringDoc OpenAPI** per documentazione API (Swagger UI).

2.3.2 Front-end

- **Angular 21** con standalone components.
- **@angular/google-maps** per integrazione mappa.
- **RxJS** per la gestione asincrona.
- **Vitest** per unit test.

2.3.3 DevOps e strumenti

- **Maven** per build e gestione dipendenze.
- **Docker** per containerizzazione.
- **JaCoCo** per coverage.
- **SonarQube** per analisi qualità del codice back-end.

2.4 Deployment e infrastruttura

Il sistema è pensato per un deployment su infrastruttura cloud, con particolare riferimento ad **AWS**.

2.4.1 Back-end e database

Il back-end **Spring Boot** è eseguito su una **Virtual Machine AWS** (es. istanza EC2) che ospita anche il database **PostgreSQL/PostGIS** utilizzato in produzione. In questo scenario:

- **L'applicazione Spring Boot** è deployata come container Docker sulla VM, garantendo l'isolamento dell'ambiente di esecuzione e la coerenza tra sviluppo e produzione.
- **PostgreSQL/PostGIS** è eseguito come container Docker sulla stessa VM. Questa scelta semplifica la gestione delle estensioni spaziali e facilita le operazioni di backup e aggiornamento.
- l'applicazione accede al database tramite credenziali configurate in variabili d'ambiente o file di configurazione esterni.

Questa configurazione semplifica la gestione iniziale dell'infrastruttura, mantenendo comunque la possibilità di evolvere verso una separazione tra application server e database server o verso servizi gestiti (ad es. Amazon RDS) in fasi successive.

2.4.2 Storage delle immagini

Le immagini (foto immobili e foto profilo agenti) non risiedono sulla VM, ma sono caricate in un **bucket Amazon S3** dedicato. L'applicazione:

- riceve i file dal front-end tramite endpoint REST;
- li inoltra al bucket S3 tramite **StorageService**, che incapsula le API AWS;
- memorizza nel database solo i riferimenti (URL e metadati), mantenendo il back-end stateless rispetto ai contenuti binari.

Questo approccio consente di:

- ridurre il carico di I/O sulla VM;
- scalare indipendentemente l'application server e lo storage;
- semplificare il backup e la replica dei contenuti multimediali.

2.5 Schema di persistenza dati

Dal punto di vista infrastrutturale, la persistenza dei dati è articolata su due piani principali:

- **Dati applicativi strutturati** (utenti, agenzie, immobili, recensioni, ecc.) memorizzati in un database relazionale.
- **Contenuti multimediali** (foto degli immobili, foto profilo agenti) memorizzati come file in uno storage oggetti esterno.

In ambiente di sviluppo viene utilizzato **H2** in-memory in modalità PostgreSQL per velocizzare il ciclo di sviluppo e testing, mentre in produzione i dati sono persistiti in un database **PostgreSQL/PostGIS** eseguito su una **Virtual Machine AWS** dedicata. Il back-end Spring Boot si connette a tale istanza tramite JDBC, sfruttando PostGIS per la gestione delle coordinate geografiche (ricerca per raggio, visualizzazione su mappa).

Le foto degli immobili e le immagini di profilo non sono salvate direttamente nel database, ma in un **bucket S3** dedicato. Il back-end interagisce con S3 tramite un componente **StorageService** che incapsula:

- il caricamento dei file (upload) verso il bucket;
- la generazione e gestione degli URL pubblici o firmati;
- l'eventuale cancellazione o aggiornamento delle immagini.

Nel database vengono quindi memorizzati solo i **metadati** (es. URL, nome file, ordine nella galleria) necessari a collegare ciascun record applicativo al corrispondente file nel bucket S3. In questo modo:

- si mantiene il database snello e performante;
- si delega la scalabilità e l'affidabilità dello storage dei file a S3;
- si semplifica il deploy del back-end, che resta stateless rispetto ai contenuti multimediali.

A livello di modello concettuale, le principali entità sono:

- **User**:
 - Attributi: `id`, `email`, `password_hash`, `first_name`, `last_name`, `registered_at`, `is_active`, `auth_provider`.
 - Relazioni: multi-a-uno con **Agency**; multi-a-molti con **Role**.
- **Agent** (sottoclasse di **User**):
 - Attributi addizionali: biografia, foto profilo (URL S3), data di nascita, numero di telefono.
- **Agency**:
 - Attributi: `id`, `name`, `email`, `phone`, `address`, `created_at`.
 - Relazioni: uno-a-molti con utenti/agent; uno-a-molti con **property**.
- **Property**:
 - Attributi: titolo, descrizione, prezzo, tipo (SALE/RENT), metratura, stanze, piano, bagni, condizione, anno, classe energetica, indirizzo, città, posizione geografica (tipo spaziale PostGIS).
 - Relazioni: multi-a-uno con **Agent** e **Agency**; uno-a-molti con **PropertyPhoto**; multi-a-molti con **Amenity**.
- **Amenity**: `id`, `name` (univoco), relazione multi-a-molti con **property**.
- **PropertyPhoto**: `id`, `url` (verso S3), eventuali metadati (ordine, descrizione), FK a **property**.
- **Review**: `id`, `score`, `comment`, `created_at`, FK a **Agent** e **User**.

2.6 Design dell'interfaccia utente

Principi generali:

- **Navigazione chiara:** navbar fissa con accesso a Home, Properties, Agents, Profile/Login.
- **Home page:** sezione *hero* con testo introduttivo e box di ricerca rapida.
- **Ricerca/listing:** form filtri in alto, lista di immobili in card e mappa affiancata.
- **Dettaglio immobile:** galleria foto, dati essenziali, mappa, contatti agente/agenzia.
- **Login/Registrazione:** layout a due colonne (branding + form), pulsanti social login, messaggi di validazione chiari.
- **Profilo utente/agente:** pagina con card dati personali e, per agenti, colonna dedicata a foto/biografia.

2.7 Diagramma delle classi di design

Il diagramma delle classi di design rappresenta:

- Gerarchia utenti: **User** e sottoclasse **Agent**.
- Dominio immobiliare: **Property**, **Agency**, **Amenity**, **PropertyPhoto**, **Review**.
- Relazioni tra servizi e repository: **PropertyService**, **AgentService**, **AuthService** con i rispettivi ***Repository**.

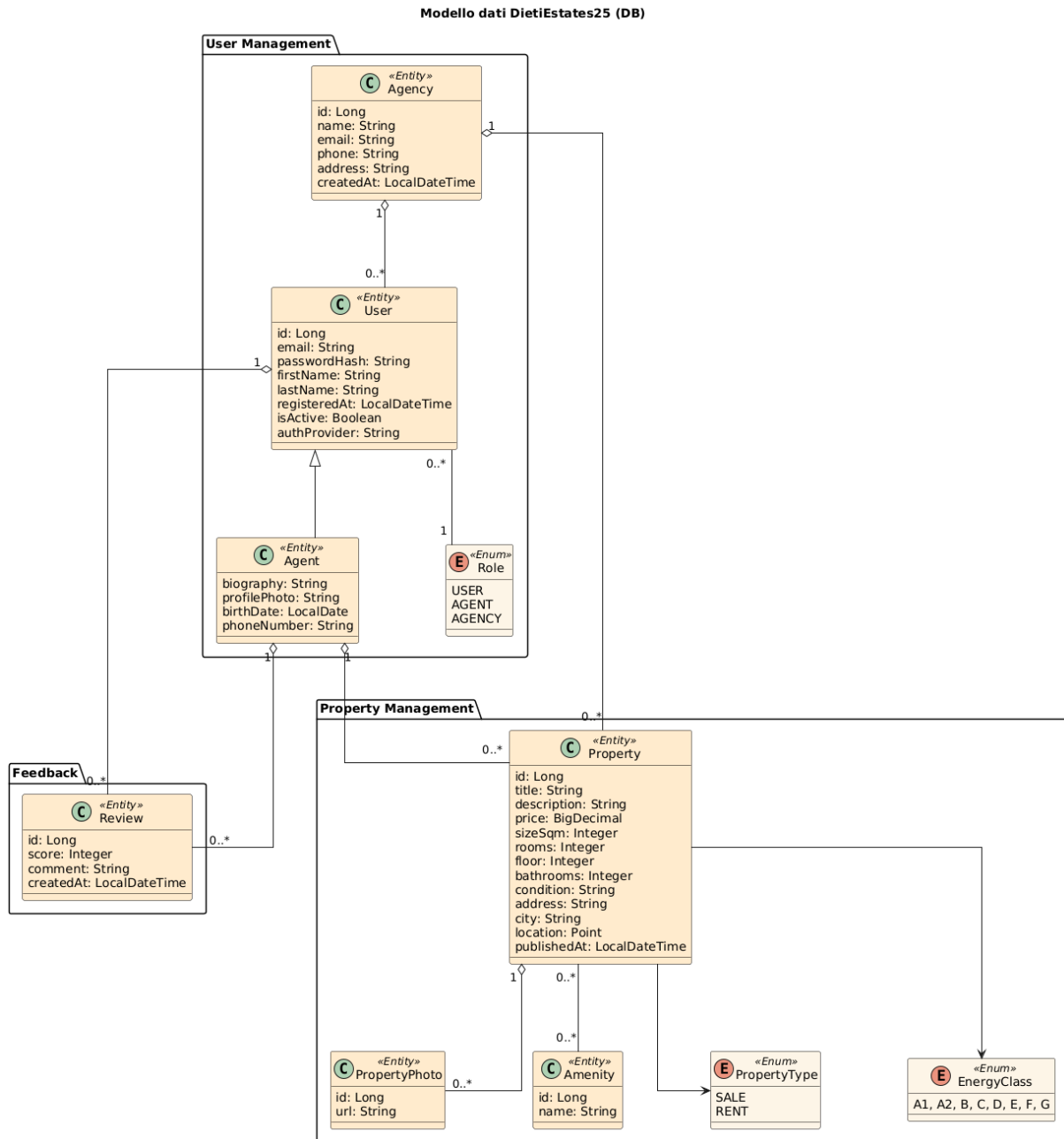


Figura 2.1: Diagramma delle classi di Design

2.7.1 Diagrammi Architeturali

Di seguito viene rappresentata l'architettura del BackEnd con diagrammi di dettaglio per il pacchetto Controller e Service.

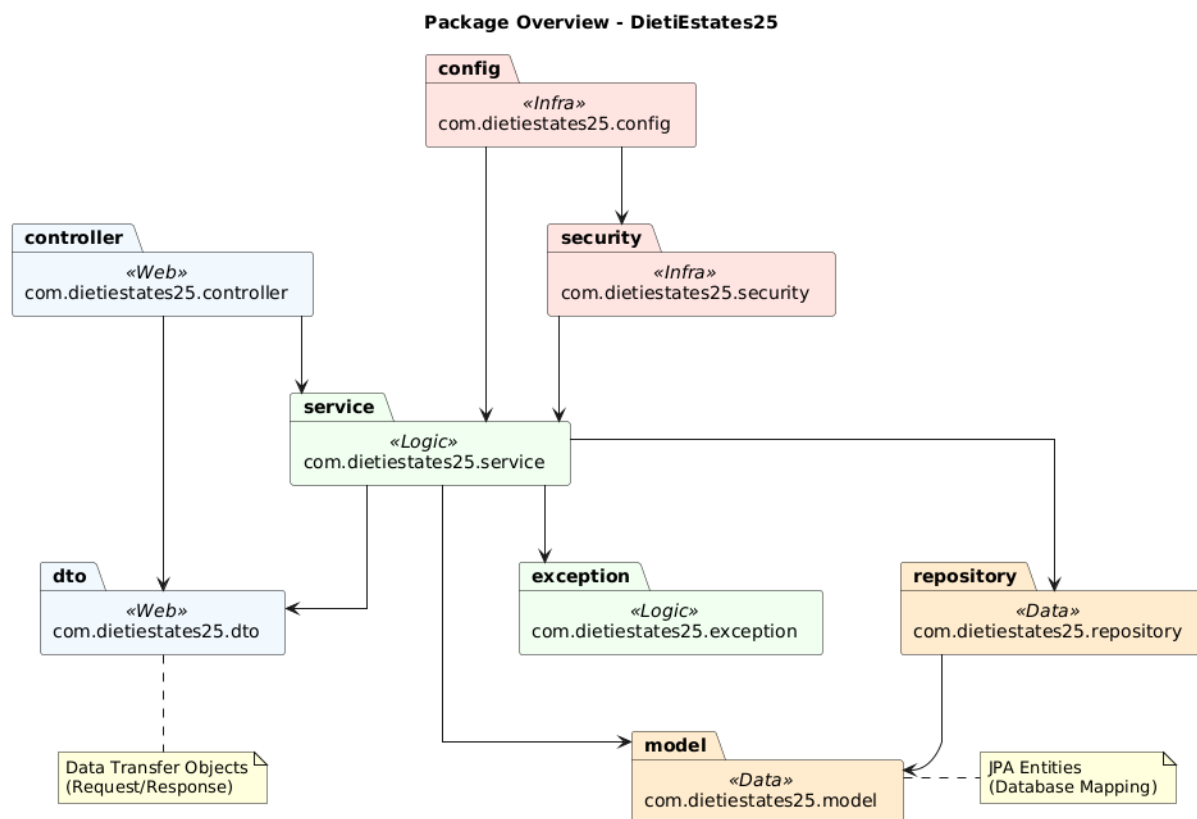


Figura 2.2: System Overview

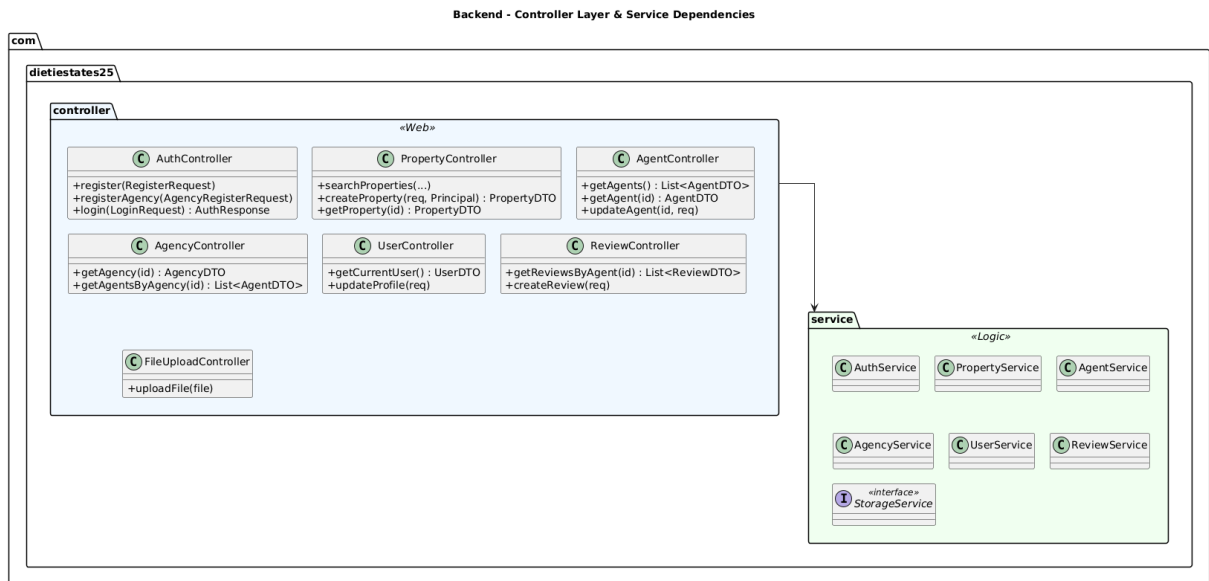


Figura 2.3: Detailed Controller View

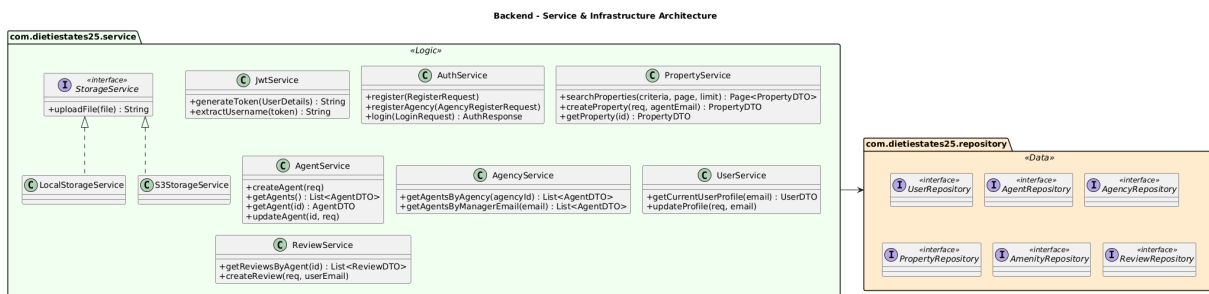


Figura 2.4: Detailed Service View

2.8 Diagrammi di sequenza

Per i casi d'uso UC06, UC08, UC10 e UC12 sono definiti i seguenti diagrammi di sequenza.

2.8.1 Sequence Diagram: UC06

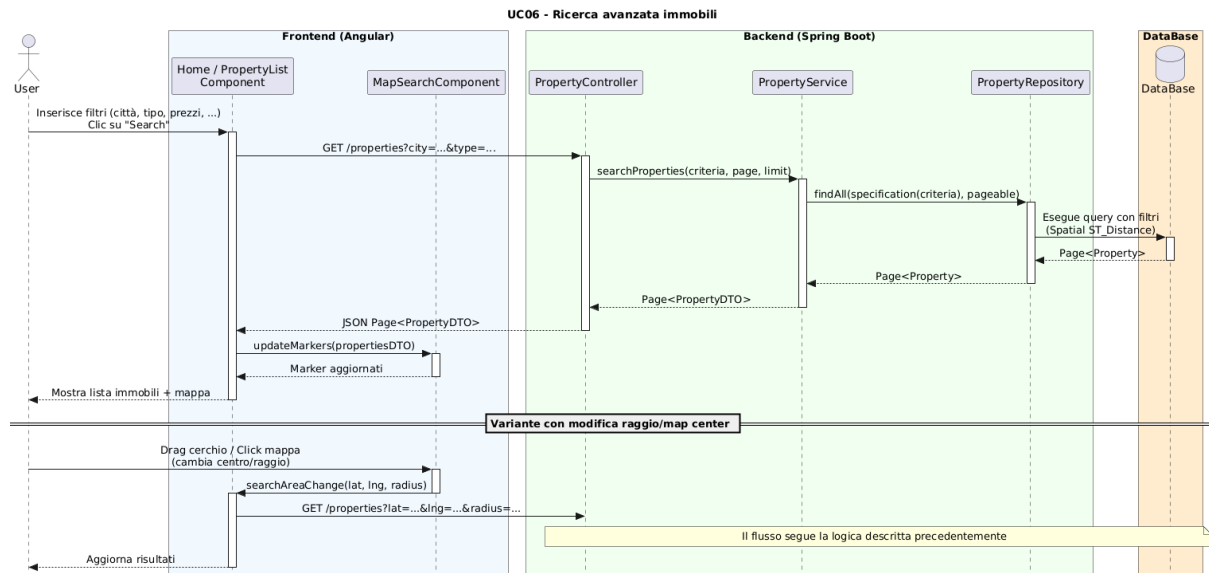


Figura 2.5: Sequence Diagram: UC06

2.8.2 Sequence Diagram: UC08

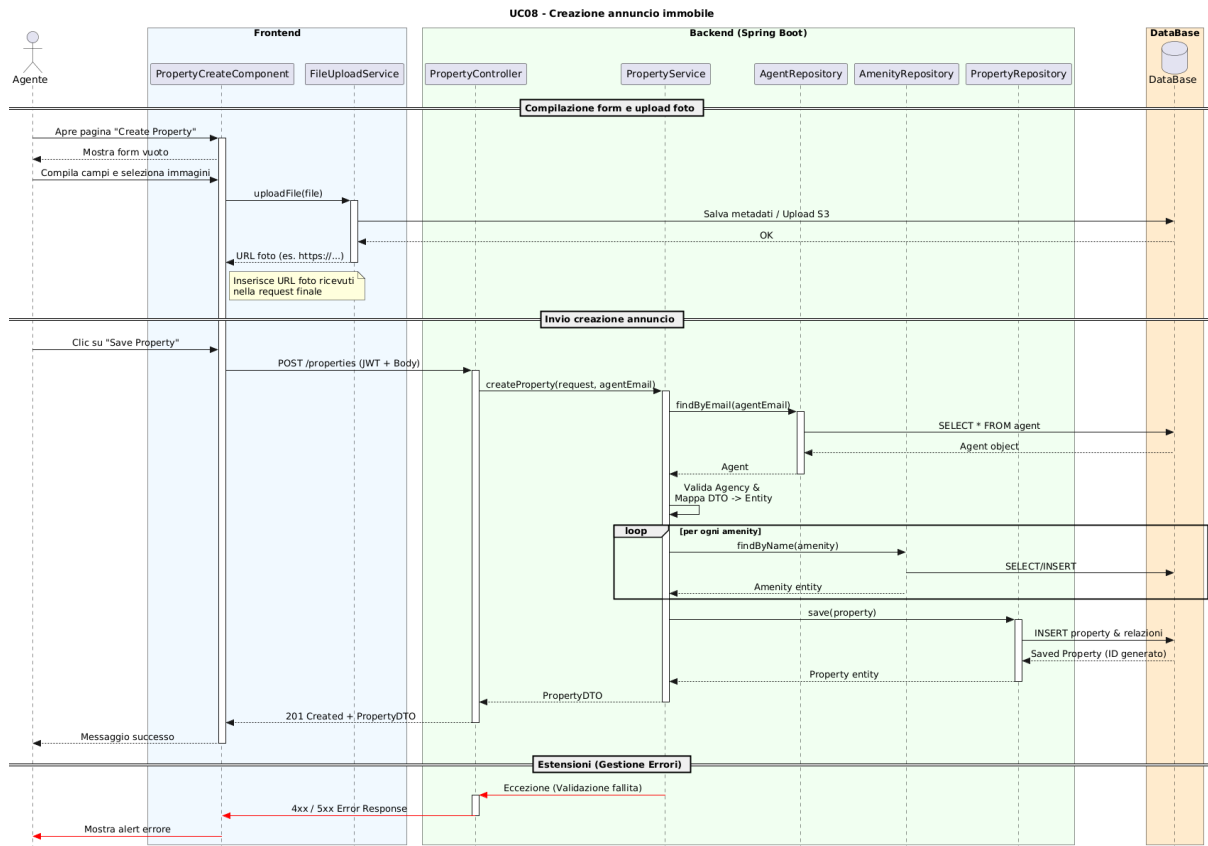


Figura 2.6: Sequence Diagram: UC08

2.8.3 Sequence Diagram: UC10

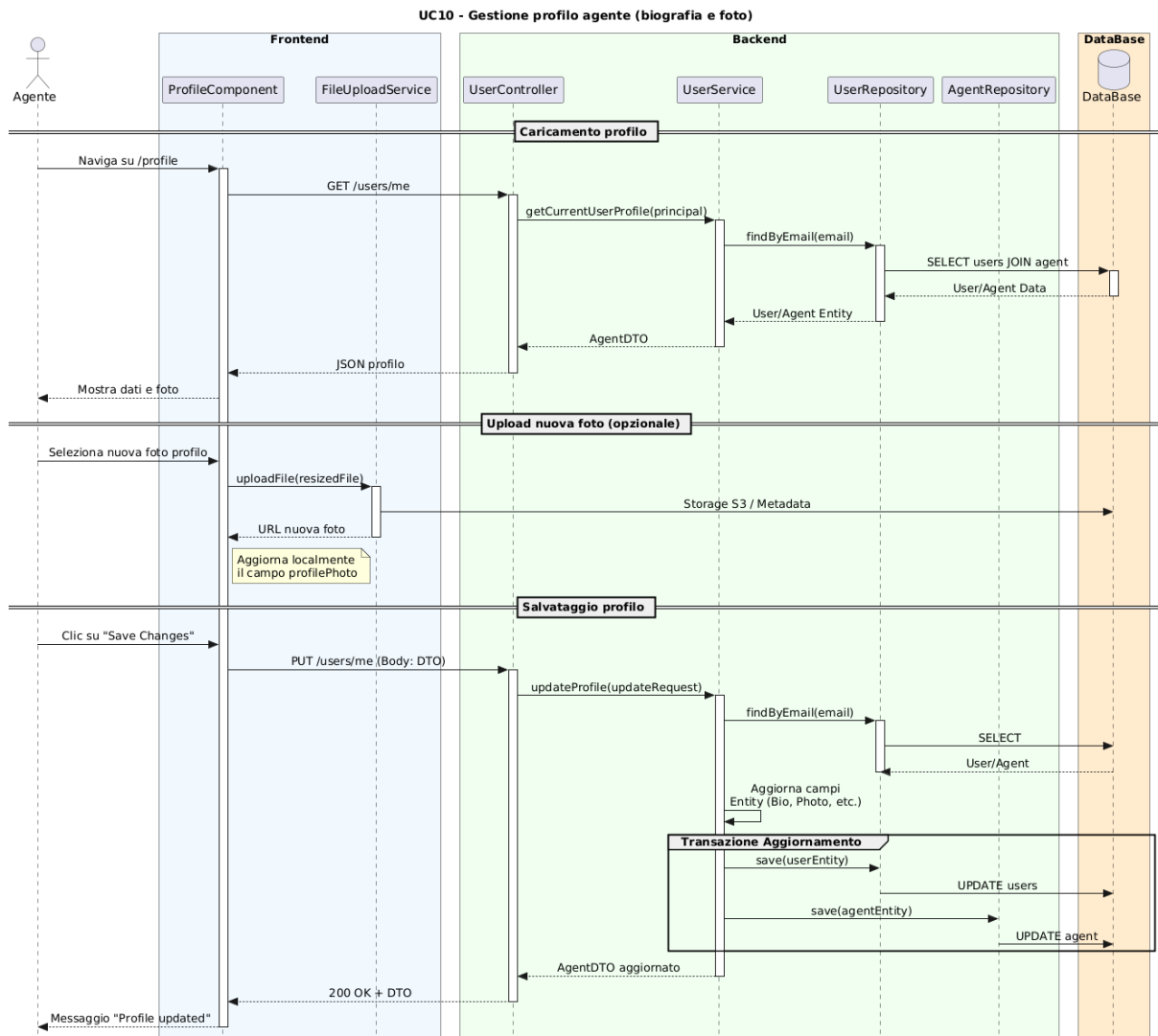


Figura 2.7: Sequence Diagram: UC10

2.8.4 Sequence Diagram: UC12

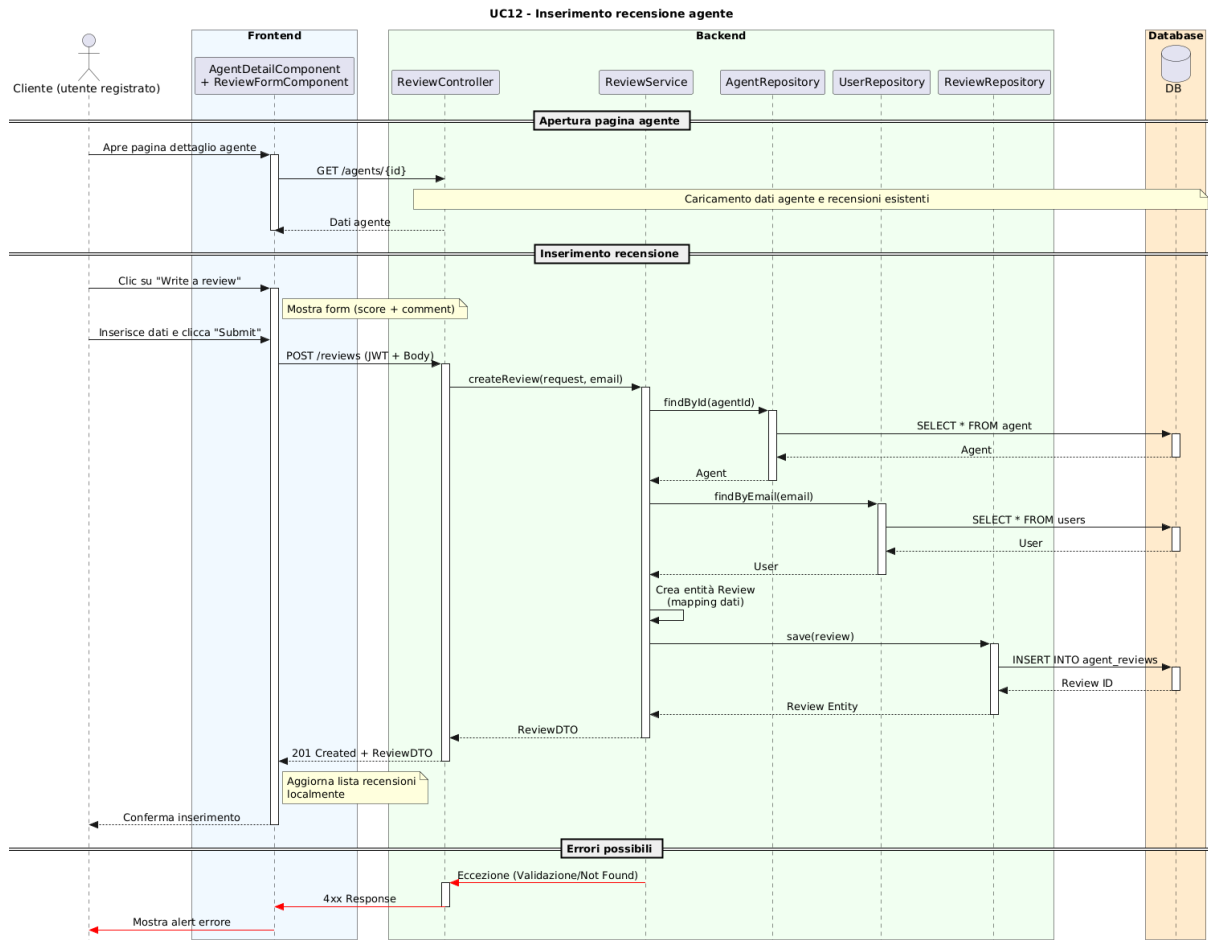


Figura 2.8: Sequence Diagram: UC12

2.9 Gestione del versioning

Il progetto utilizza **Git** come sistema di versioning distribuito, con repository ospitato su **GitHub**. Questa scelta garantisce:

- tracciabilità completa delle modifiche al codice;
- collaborazione efficace tra i membri del team;
- possibilità di gestire branch separati per sviluppo, testing e produzione;
- integrazione con strumenti di CI/CD e analisi della qualità del codice.

2.9.1 Statistiche del repository

Il repository GitHub mostra un'attività di sviluppo costante durante il ciclo di vita del progetto. Le metriche principali includono:

- **Numero totale di commit:** 71
- **Numero di contributors:** 1

I commit seguono la convenzione dei conventional commits per facilitare la comprensione della storia del progetto e per la generazione automatica di changelogs.



Figura 2.9: Statistiche del repository GitHub

2.10 Analisi della qualità del codice

Per garantire la qualità e la manutenibilità del codice back-end, il progetto utilizza **SonarQube**, una piattaforma di analisi statica del codice che valuta:

- **Code smells**: problemi di design e architettura che possono compromettere la manutenibilità;
- **Bug**: errori potenziali che potrebbero causare comportamenti inattesi;
- **Vulnerabilità di sicurezza**: problemi di sicurezza che potrebbero essere sfruttati da attaccanti;
- **Coverage**: percentuale di codice coperta da test unitari;
- **Duplicazione**: presenza di codice duplicato che può essere refactorizzato;
- **Debito tecnico**: tempo stimato necessario per risolvere tutti i problemi rilevati.

2.10.1 Report SonarQube

L'analisi SonarQube sul codice back-end Spring Boot evidenzia:

- un livello di qualità complessivo conforme agli standard del progetto;
- presenza di test unitari con copertura adeguata per i componenti critici (service layer, controller principali);
- assenza di vulnerabilità critiche o ad alto rischio.

Il report fornisce metriche dettagliate per ogni componente (classi, metodi, linee di codice) e suggerimenti specifici per il miglioramento continuo della qualità del codice.

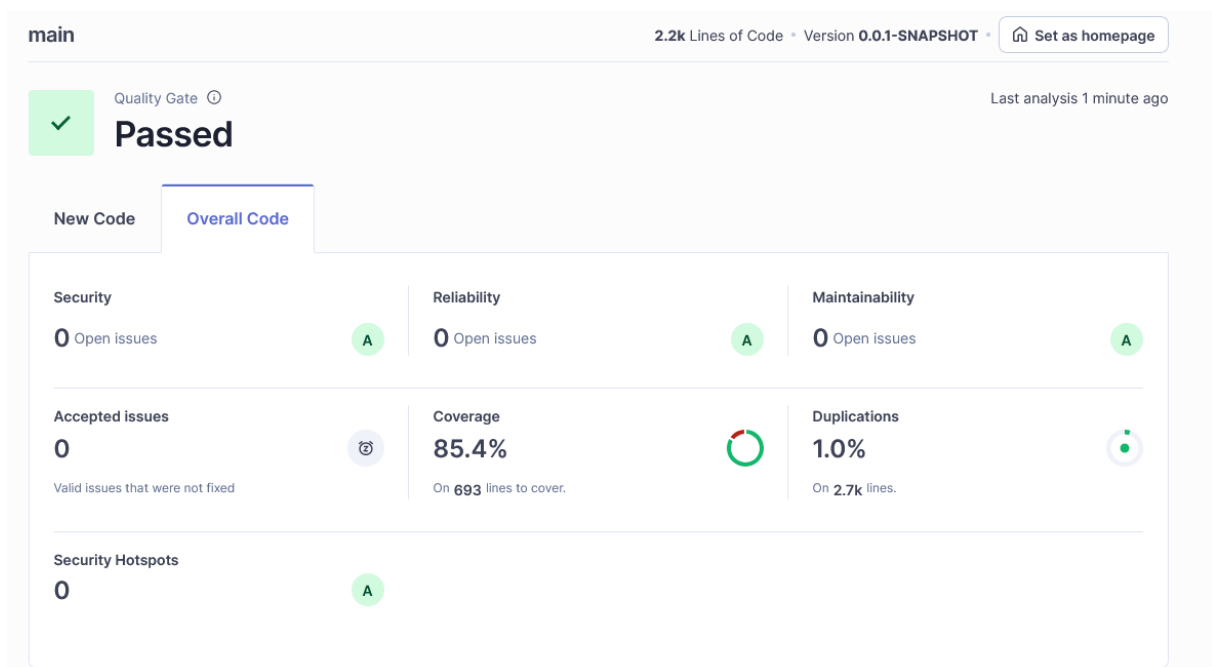


Figura 2.10: Report di qualità del codice SonarQube

Capitolo 3

Testing e valutazione dell'usabilità

3.1 Strategia di unit testing

Il back-end utilizza **JUnit 5** come framework di testing e **Mockito** per il mocking delle dipendenze. La strategia adottata segue il pattern *Arrange-Act-Assert* (AAA) e si basa su:

- **Isolamento delle unità:** ogni test isola la classe sotto test (Service) mockando le dipendenze esterne (Repository, altri Service, componenti infrastrutturali).
- **Test delle responsabilità:** ogni test verifica un singolo comportamento o scenario, garantendo che i test siano focalizzati e facilmente debuggabili.
- **Copertura di casi positivi e negativi:** per ogni metodo critico vengono testati sia il percorso di successo che i percorsi di errore (eccezioni attese).

3.1.1 Classi di equivalenza e criteri di copertura

Per la progettazione dei test vengono identificate **classi di equivalenza** sui parametri di ingresso di ciascun metodo. Una classe di equivalenza raggruppa valori di input che producono lo stesso comportamento atteso del sistema. I test vengono progettati per coprire almeno un rappresentante per ogni classe valida e invalida.

Criteri di copertura strutturale

I test mirano a raggiungere:

- **Copertura delle istruzioni:** ogni istruzione del codice viene eseguita almeno una volta.
- **Copertura dei branch:** ogni ramo condizionale (if/else, switch) viene percorso sia nel caso `true` che `false`.
- **Copertura dei path:** per metodi complessi con più condizioni annidate, vengono testate combinazioni significative di percorsi di esecuzione.

3.1.2 Esempi di metodi testati con classi di equivalenza

`searchProperties(criteria, page, limit)` in `PropertyService`

Questo metodo implementa la ricerca avanzata con filtri multipli e supporto per ricerca geografica. Le classi di equivalenza identificate sono:

Parametro	Classi di equivalenza
criteria	• CE1: criteri vuoti/null (ricerca senza filtri) • CE2: criteri con filtri testuali (città, tipo, classe energetica) • CE3: criteri con filtri numerici validi (prezzo, stanze, metratura, piano, bagni) • CE4: criteri con filtri geografici (latitudine, longitudine, raggio) • CE5: combinazione di filtri multipli (testuali + numerici + geografici) • CE6: filtri numerici incoerenti ($\text{minPrice} > \text{maxPrice}$, $\text{minSize} > \text{maxSize}$) – <i>classe invalida</i>
page	• CE7: valore valido (≥ 0) • CE8: valore negativo – <i>classe invalida</i>
limit	• CE9: valore valido (> 0) • CE10: valore ≤ 0 – <i>classe invalida</i>

Tabella 3.1: Classi di equivalenza per `searchProperties`

Casi di test implementati:

- `searchProperties_AllFilters_CallsRepo`: verifica il percorso con tutti i filtri popolati (CE5), assicurando che la `Specification` venga costruita correttamente e che il repository venga chiamato con i parametri attesi.
- `searchProperties_GeoFilter_CallsRepo`: verifica il percorso con filtri geografici (CE4), testando la costruzione della query PostGIS per il calcolo della distanza.

Copertura strutturale: i test coprono i branch principali della costruzione della `Specification` (presenza/assenza di ciascun filtro) e verificano che i predicati vengano combinati correttamente tramite `and()`.

`createProperty(request, agentEmail)` in `PropertyService`

Metodo critico per la creazione di immobili. Le classi di equivalenza sono:

Parametro	Classi di equivalenza
request.type	• CE1: valore valido ("SALE", "RENT") • CE2: valore invalido ("INVALID_TYPE") – <i>classe invalida</i>
agentEmail	• CE3: email esistente nel repository • CE4: email non esistente – <i>classe invalida</i> • CE5: agente esistente ma senza agenzia associata – <i>classe invalida</i>
request.amenities	• CE6: lista vuota/null • CE7: lista con amenity esistenti nel repository • CE8: lista con amenity non esistenti – <i>classe invalida</i> (gestita a runtime)
request.photos	• CE9: lista vuota/null • CE10: lista con URL validi

Tabella 3.2: Classi di equivalenza per `createProperty`

Casi di test implementati:

- `createProperty_ValidData_SavesProperty`: verifica il percorso di successo (CE1, CE3, CE7, CE10), controllando che l'entità venga salvata con tutti i campi mappati correttamente e che le relazioni con `Amenity` e `Agency` siano stabilite.
- `createProperty_InvalidType_ThrowsException`: verifica che venga sollevata un'eccezione per tipo invalido (CE2).
- `createProperty_AgentNotFound_ThrowsException`: verifica l'eccezione per agente non trovato (CE4).
- `createProperty_AgentNoAgency_ThrowsException`: verifica l'eccezione per agente senza agenzia (CE5).

Copertura strutturale: i test coprono tutti i branch condizionali del metodo (controllo esistenza agente, controllo presenza agenzia, validazione tipo, gestione amenities e foto).

`register(RegisterRequest request)` in `AuthService`

Metodo per la registrazione di nuovi utenti. Le classi di equivalenza sono:

Parametro	Classi di equivalenza
<code>request.email</code>	• CE1: email non esistente nel repository (nuova registrazione) • CE2: email già esistente – <i>classe invalida</i>
<code>request.password</code>	• CE3: password valida (non vuota, lunghezza minima rispettata) • CE4: password invalida (vuota, troppo corta) – <i>classe invalida</i> (validata a livello di DTO/Controller)
<code>request.firstName</code> , <code>request.lastName</code>	• CE5: valori non nulli e non vuoti • CE6: valori nulli o vuoti – <i>classe invalida</i> (validata a livello di DTO)

Tabella 3.3: Classi di equivalenza per `register`

Casi di test implementati:

- `register_NewEmail_SavesUser`: verifica il percorso di successo (CE1, CE3, CE5), controllando che l'utente venga salvato con password codificata e ruolo `USER` assegnato.
- `register_ExistingEmail_ThrowsException`: verifica che venga sollevata un'eccezione con messaggio appropriato per email già registrata (CE2).

Copertura strutturale: i test coprono il branch principale (controllo esistenza email) e verificano lo stato indotto sul repository (salvataggio con attributi corretti).

`login(LoginRequest request)` in `AuthService`

Metodo per l'autenticazione degli utenti. Le classi di equivalenza sono:

Parametro	Classi di equivalenza
<code>request.email</code> , <code>request.password</code>	• CE1: credenziali corrette (email esistente, password corrispondente) • CE2: credenziali errate (password non corrispondente) – <i>classe invalida</i> • CE3: utente non esistente (email non presente nel repository) – <i>classe invalida</i>

Tabella 3.4: Classi di equivalenza per `login`

Casi di test implementati:

- `login_ValidCredentials_ReturnsToken`: verifica il percorso di successo (CE1), controllando che venga generato un JWT valido e che la risposta contenga i dati utente corretti.
- `login_InvalidCredentials_ThrowsException`: verifica che venga sollevata `BadCredentialsException` per password errata (CE2).
- `login_UserNotFound_ThrowsException`: verifica l'eccezione per utente non trovato (CE3).

Copertura strutturale: i test coprono tutti i branch del metodo (autenticazione riuscita, fallita per credenziali errate, fallita per utente inesistente).

`createAgent(request)` in `AgentService`

Metodo per la creazione di agenti da parte del manager di agenzia. Le classi di equivalenza sono:

- **CE1**: manager autenticato esistente con agenzia associata (caso valido).
- **CE2**: manager non trovato nel repository – *classe invalida*.
- **CE3**: manager esistente ma senza agenzia associata – *classe invalida*.

Casi di test implementati:

- `createAgent_ValidRequest_SavesAgent`: verifica il percorso di successo (CE1), controllando che l'agente venga salvato con agenzia corretta e ruolo `AGENT`.
- `createAgent_ManagerNotFound_ThrowsException`: verifica l'eccezione per manager non trovato (CE2).
- `createAgent_ManagerHasNoAgency_ThrowsException`: verifica l'eccezione per manager senza agenzia (CE3).

3.1.3 Verifica dei risultati e dello stato

Per ciascun test vengono verificati:

- **Valori di ritorno:** il metodo restituisce il valore atteso (DTO, entità, o void con effetti collaterali verificati).
- **Stato indotto sul repository:** tramite `verify()` di Mockito si controlla che i metodi del repository vengano chiamati con i parametri corretti e il numero di volte atteso.
- **Eccezioni attese:** per i casi negativi, si verifica che venga sollevata l'eccezione corretta con `assertThrows()` e, quando rilevante, che il messaggio di errore sia appropriato.

3.1.4 Copertura del codice

La copertura del codice viene misurata tramite **JaCoCo** durante l'esecuzione dei test Maven. L'obiettivo è raggiungere una copertura superiore all'80% per le classi di servizio critiche (**PropertyService**, **AuthService**, **AgentService**) e per i componenti di sicurezza (**JwtService**, **JwtAuthenticationFilter**).

3.2 Testing di integrazione e API

I test di integrazione verificano il comportamento end-to-end delle API REST, testando l'interazione tra Controller, Service e Repository (o mock del repository, a seconda del livello di isolamento desiderato).

3.2.1 Test dei Controller

I test dei controller utilizzano **Spring Test** con **@WebMvcTest** per caricare solo il contesto web necessario, mockando i Service layer tramite **@MockBean**. Questo approccio:

- isola il layer di presentazione dal layer di business;
- permette di testare il mapping HTTP (URI, metodi, parametri di query, body delle richieste);
- verifica la serializzazione/deserializzazione JSON dei DTO;
- valida l'applicazione dei filtri di sicurezza e la gestione delle eccezioni a livello di controller.

Esempi di test implementati

- **PropertyControllerTest.shouldSearchProperties**: verifica che l'endpoint **GET /properties** con parametri di query venga mappato correttamente, che il Service venga chiamato con i criteri corretti e che la risposta HTTP 200 contenga il JSON atteso.
- **AuthControllerTest.shouldRegisterUserSuccessfully**: verifica che **POST /auth/register** con body valido restituisca HTTP 201.
- **AuthControllerTest.shouldReturnBadRequestOnInvalidRegister**: verifica la validazione a livello di controller (HTTP 400 per dati invalidi).
- **AuthControllerTest.shouldReturnConflictIfEmailExists**: verifica la gestione delle eccezioni di business (HTTP 409 per email già esistente).
- **AuthControllerTest.shouldLoginSuccessfully**: verifica il percorso di successo del login (HTTP 200 con token JWT).
- **AuthControllerTest.shouldReturnNotFoundIfUserNotFound**: verifica la gestione di utenti non trovati (HTTP 404).

3.2.2 Criteri di copertura per i test di integrazione

I test di integrazione coprono:

- **Mapping degli endpoint:** correttezza dell'URI, del metodo HTTP (GET, POST, PUT, DELETE) e dei parametri (path variables, query parameters, request body).
- **Codici di stato HTTP:** verifica dei codici attesi (200, 201, 400, 401, 404, 409, 500) per ogni scenario.
- **Struttura delle response:** validazione della struttura JSON delle risposte tramite `jsonPath()` di Spring Test.
- **Sicurezza:** verifica che gli endpoint protetti richiedano autenticazione e che gli endpoint pubblici siano accessibili senza token.
- **Gestione degli errori:** verifica che le eccezioni vengano trasformate correttamente in risposte HTTP con codici e messaggi appropriati tramite `GlobalExceptionHandler`.

3.2.3 Test di integrazione con database

Per i test che richiedono un database reale (ad esempio, test end-to-end completi), viene utilizzato un database H2 in-memory configurato tramite `@SpringBootTest` con profilo di test. Questi test verificano:

- la persistenza corretta delle entità;
- l'integrità referenziale;
- le query complesse (es. ricerca geografica con PostGIS).

Tuttavia, per la maggior parte dei test di integrazione, si preferisce mockare il repository per mantenere i test veloci e isolati.

3.3 Valutazione dell'usabilità

La valutazione dell'usabilità di DietiEstates25 è stata svolta tramite due approcci complementari: (i) ispezione esperta basata sulle 10 euristiche di Nielsen, per identificare problemi sistematici dell'interfaccia; (ii) sessioni di user testing strutturate su task rappresentativi, con raccolta di metriche quantitative e feedback qualitativo.

3.3.1 Expert reviews / inspections (Nielsen Heuristics)

Metodo Heuristic Evaluation basata sulle **10 Usability Heuristics** di Nielsen.

Valutatori 3 valutatori indipendenti (ispezione interna) su una build stabile.

Output Lista consolidata di issue e raccomandazioni, con priorità (alta/media/bassa).

Risultati principali (sintesi consolidata)

- **H1 – Visibilità dello stato del sistema:** feedback positivi su caricamenti e stati di salvataggio degli annunci.
- **H2 – Corrispondenza con il mondo reale:** terminologia di dominio coerente (Mq, Classe Energetica, Rating).
- **H3 – Controllo e libertà dell'utente:** presente il reset dei filtri e la possibilità di annullare l'inserimento di un annuncio prima della pubblicazione.
- **H4 – Coerenza e standard:** componenti UI consistenti; le icone per le recensioni e i form di inserimento seguono pattern attesi.
- **H7 – Flessibilità ed efficienza:** il sistema è intuitivo per i nuovi utenti, ma la ricerca avanzata richiede un attimo prima di essere compresa.
- **H9 – Recupero dagli errori:** messaggi di validazione nei form (prezzo mancante, formato email errato) potrebbero essere migliorati.

3.3.2 Esperimento con utenti

Obiettivo

Valutare l'efficacia e la soddisfazione durante la gestione attiva dei contenuti (creazione e valutazione) oltre alla semplice consultazione:

- ricerca avanzata e filtraggio immobili;
- inserimento di nuovi annunci nel database (lato venditore/agente);
- valutazione dell'operato degli agenti tramite il sistema di recensioni.

Partecipanti

- **Campione:** N=5 utenti (potenziali venditori/acquirenti).
- **Età:** 24–45 anni.
- **Profilo:** competenze digitali eterogenee; familiarità generica con portali immobiliari.

Task

- **Task 1 (Search & Filter):** trovare un immobile a Napoli con 4 stanze, circa 80 m² e prezzo nell'intorno di 200.000€.
- **Task 2 (Details):** dal primo risultato, individuare *classe energetica* e verificare la presenza di *garage*.
- **Task 3 (Create Property):** creare un nuovo annuncio immobiliare inserendo dati tecnici, descrizione e prezzo, completando la pubblicazione.
- **Task 4 (Agent Review):** individuare il profilo di un agente e pubblicare una recensione con feedback testuale e valutazione numerica.

3.3.3 Risultati

Sintesi quantitativa

Tutti gli utenti hanno completato con successo i task assegnati senza richiedere l'intervento del moderatore.

Metrica	Task 1	Task 2	Task 3	Task 4	Overall
Success Rate	100%	100%	100%	100%	100%

Tabella 3.5: Sintesi quantitativa dell'efficacia per task

SUS Il punteggio SUS medio è pari a **88/100**, indicativo di una usabilità *eccellente*. Nonostante tutti i partecipanti abbiano completato i task con successo, sono emersi margini di miglioramento relativi alla comunicazione del sistema verso l'utente.

Sintesi qualitativa

Punti di forza osservati

- **Workflow di creazione fluido:** il form di inserimento immobile è stato percepito come ben strutturato, evitando il senso di sopraffazione nonostante la mole di dati richiesti.
- **Sistema di recensioni immediato:** l'interfaccia di rating e commento è risultata familiare, rapida e priva di attriti cognitivi.
- **Efficacia dei risultati:** la precisione dei parametri di ricerca permette di individuare l'immobile desiderato con pochissimi passaggi.

Criticità ricorrenti

- **Chiarezza dei messaggi di errore:** in alcuni casi di validazione fallita (es. formati dati specifici), i messaggi restituiti dal sistema sono stati percepiti come troppo tecnici o non sufficientemente esplicativi sulla risoluzione del problema.
- **Densità visiva della ricerca:** la sezione dei filtri avanzati, pur essendo completa, presenta una densità di controlli che può inizialmente disorientare l'utente meno esperto.
- **Feedback di conferma:** dopo la pubblicazione di un annuncio o l'invio di una recensione, gli utenti hanno suggerito la necessità di un feedback visivo più marcato (es. una modale di successo o un redirect esplicito) per confermare l'avvenuta operazione.

3.3.4 Conclusioni e raccomandazioni

In base ai risultati del testing e all'analisi delle euristiche, le priorità di miglioramento individuate sono:

- **Alta: Refactoring della messaggistica di errore.** Tradurre i codici di errore del back-end in suggerimenti testuali "user-friendly" che indichino chiaramente come correggere i dati inseriti.
- **Media: Ottimizzazione UI dei filtri.** Riorganizzare il layout della ricerca avanzata (es. tramite sezioni collassabili) per ridurre il carico cognitivo iniziale.
- **Bassa: Potenziamento dei feedback visivi.** Introdurre notifiche "toast" o modali di conferma più evidenti al completamento dei flussi principali (creazione immobile, invio recensione, modifica profilo).